

Clearspace Algorithms and Data Structures Enhancement Narrative

Payton Castle
CS 499 Milestone Three
September 23, 2026

Artifact Background

For this milestone, I enhanced Clearspace to handle deep folder structures consistently and calculate folder sizes efficiently. I created Clearspace in August 2026 as a Windows file manager using C# and Windows Presentation Foundation. It supports navigation, search, tags, and file operations. This enhancement builds on the version with my Artifact 1 software engineering changes already completed. It focuses on algorithms and data structures through improved traversal, a disk usage analyzer, and more reliable index recovery.

Reason for Selection and Improvements

I selected Clearspace because it connects algorithm choices to problems I encounter while using the application. A drive can contain millions of entries, and a folder can have many levels of subfolders. The application needs to produce useful results without making navigation freeze. This artifact demonstrates how I evaluate completeness, execution time, memory use, and responsiveness together.

The previous index builder stopped after 32 folder levels, while fallback search stopped after 24. Those different limits could produce different results for the same location. Indexing now uses an explicit stack, and fallback search uses queues without arbitrary depth limits. Cancellation checks let background work stop when it is no longer needed. Both paths apply the same policy for folder links and supported cloud placeholders, reducing inconsistent coverage and avoiding link cycles.

Folder sizes are calculated with a bottom-up pass over the indexed parent relationships. Each subtree total is added to its parent once, giving $O(n)$ aggregation time. Repeatedly walking the ancestors of every file could instead require $O(n \text{ times } d)$, where d is folder depth. Parallel arrays hold totals and hierarchy links, keeping the representation compact. The treemap also caches geometry and stores subtree ranges so that off-screen branches can be skipped efficiently.

Index health is tracked separately for each drive. If changes are missed on one drive, other healthy drives can continue using indexed search. A replacement index receives changes recorded during its scan before publication. The indexing page makes this behavior visible by showing indexed entries, storage usage, scan activity, and reasons for waiting. These interface additions help users understand the underlying work, while the main algorithmic improvement remains consistent traversal and efficient aggregation.

Course Outcomes and Enhancement Scope

The completed work meets the traversal and folder-size goals described in my code review. It provides direct evidence for the course outcome about designing and evaluating computing solutions using algorithmic principles while managing design tradeoffs. The stack and queue traversals improve coverage, the reverse aggregation pass avoids repeated ancestor walks, and cached layouts reduce repeated work. Each choice also has a cost: deeper traversal can take longer, arrays and caches consume memory, and stable layouts can retain less-square rectangles to preserve recognizable positions.

The enhancement also supports the outcome about using well-founded computing techniques and tools to implement useful solutions. Regression tests cover deep traversal, cancellation, cache replacement, layout properties, and concurrent index updates. The latest full Release suite passed 158 tests. The indexing explanations and implementation notes support professional technical communication, while safe link handling and validation of hierarchy and size calculations contribute to a security mindset. These are examples of progress toward the outcomes rather than proof of complete competency in every category.

My outcome emphasis remains algorithms and data structures. The scope expanded to include cache correctness, per-drive recovery, and clearer indexing feedback because those issues affected whether the planned improvements were useful in practice. Database migration remains separate from this artifact. I will use instructor feedback to refine the outcome coverage and the final ePortfolio presentation.

Reflection on Learning and Challenges

A major lesson from this work is that a fast operation still needs to return trustworthy results. Removing depth limits addressed missing files, but also increased the amount of work the application could perform. That made cancellation and memory limits more important. Caching improved repeated navigation, but required a clear rule for when stored results became stale. A snapshot based on an older index cannot safely supply totals for its replacement.

The disk viewer exposed challenges that were difficult to judge from code alone. Dense folders produced freezing and flickering, and at one point labels and outlines moved without the corresponding blocks. This showed why the camera, displayed geometry, and hit targets need to describe the same frame. The corrections and regression checks reinforced the value of testing behavior during movement and background loading, rather than checking only a static view.

The indexing page taught a similar lesson about communication. A percentage based on discovered folders can decrease when additional folders are found. An empty progress bar did not explain why scanning had stopped. Showing the reason for waiting, the automatic retry time, and a Scan now action made the state more understandable. I learned that useful feedback must explain what the system is doing and what the user can do next.

Automated tests made the changes easier to verify repeatedly, including a native 48-level folder fixture and dense synthetic rendering cases. Their limits matter too: passing tests does not establish smooth performance on every real drive. Unreadable folders and excluded links still affect coverage, and recorded file lengths can differ from physical disk usage. My next step is to incorporate instructor feedback and continue checking real-drive behavior before the final ePortfolio submission.